

Relatório técnico grafos

Parte 1:

1) NÓS: bairros do Recife:

Nós começamos criando o repositório e a estrutura base do projeto com o primeiro csv disponibilizado “bairros_recife.csv”, a partir disso baixamos a biblioteca do pandas responsável por dissolver o primeiro csv para assim torná-lo utilizável com o o “bairros_unique.csv” onde tinha os bairros e a quantidade de conexões que ele tinha

2) ARESTAS (interconexões):

Aqui nós fizemos junto com o ponto 5, calculamos a distância utilizando um geojson com os polígonos do bairro e coloquei no dataset para tratar os erros e as repetições num notebook no colab, que está na pasta /data do projeto, o csv_organizer.ipynb, nesse notebook eu também usei de um outro csv, trechologradouro.csv, que continha todas as ruas de recife, mas tinha uma repetição de linhas no csv com a mesma rua e apenas um bairro que aquela rua conectava por linha, então eu tive que conectar essas ruas num novo dataset, para assim juntar os pares de bairro que se conectam e a rua que os conectava num novo dataset que seria utilizado para as arestas do grafo, o algoritmo que foi utilizado foi não foi perfeito, então tivemos que manualmente verificar tudo, então colocamos manualmente algumas arestas que faltavam. Depois no Python em [Graph.py](#) criamos a classe Graph que seria utilizada para representar o grafo e em [io.py](#) carregamos o grafo da cidade do Recife.

3) Métricas globais e por grupo

Após fazer as interconexões nós criamos uma função na pasta [solve.py](#) para fazer a ordem o tamanho e a densidade que as três fórmulas foram dadas tanto para a cidade do recife como um todo, para as microrregiões, e para cada bairro a partir disso tivemos mais três arquivos csv que são:

- recife_global.json (ordem, tamanho, densidade)
- microrregioes.json (lista com métricas por microrregião)

-ego_bairro.csv (tabela completa por bairro)

4) Graus e rankings

E então pegamos o arquivo “bairros_unique.csv” e a partir dele calculamos os graus de cada bairro do recife(que é igual ao número de conexões que esse bairro tem) e criamos graus.csv além do bairro mais denso e bairro com maior grau e colocamos em recife.json

5) Pesos das arestas (definição criativa e consistente)

Para definir as distâncias nós tentamos fazer utilizando algoritmos que calculam a distância real entre os centróides dos bairros, utilizando de um geojson que continha os polígonos dos bairros do recife,junto com o geopandas conseguimos gerar as distâncias reais entre os centróides dos bairros que se tocavam de acordo com os polígonos do geojson, porém o algoritmo acabavam não sendo 100 por cento certo, então conseguimos mais ou menos umas 240 conexões e tivemos que utilizar um mapa da cidade para verificar as conexões que faltavam manualmente e suas distâncias nos baseamos nas distâncias que já havíamos calculados.

6) Distância entre endereços X e Y

Então criamos o arquivo “enderecos.csv” onde colocamos os endereços que decidimos com suas respectivas ruas e bairros de origem e destino , após isso criamos distancias_enderecos.csv que registra o caminho percorrido do bairro de origem ao de destino além do custo. O custo do percurso é a distância real como explicado na parte 5.

7) Transforme o percurso em árvore e mostre

Pegamos o caminho já pronto do percurso_nova_descoberta_setubal.json e só precisamos fazer as conexões e gerar no html

8) Explorações e visualizações analíticas

O histograma mostra uma distribuição assimétrica à direita, mas com cauda curta: há alguns bairros mais conectados, porém sem a presença de super-hubs extremos. A rede é moderadamente heterogênea — certos bairros são importantes para a conectividade, mas o sistema não depende de poucos nós centrais. Mesmo variando o número de bins (5, 10, 15, 20), o padrão se mantém: concentração clara nos graus médios (4–7) e poucos casos nas faixas mais altas (9–11). Visualmente, não há sinais

fortes de uma lei de potência pura; trata-se de uma malha urbana parcialmente hub-centrada, mas ainda bem distribuída.

8.2 Densidade da ego-subrede e grau médio por microrregião

O segundo painel combina duas perspectivas: o ranking dos 20 bairros com maior densidade de ego-subrede e o grau médio por microrregião. Esses bairros apresentam densidades entre 1,6 e 2,0, com a maior parte entre 1,6–1,8 e quatro bairros atingindo o valor máximo (2,0), entre eles Pau Ferro, Cacote, Soledade e Sítio dos Pintos.

A análise destaca dois tipos de centralidade:

- **Hubs locais**, que têm alta densidade de ego e fortalecem a coesão interna de suas vizinhanças, mesmo sem grau global elevado.
- **Hubs de trânsito**, mais conectados no grafo geral, mas com vizinhanças menos densas entre si.

8.3 Subgrafo dos 10 bairros com maior grau

O terceiro painel mostra o subgrafo dos bairros mais conectados: Afogados, Casa Amarela, San Martin, Arruda, Cordeiro, Graças, Curado, Boa Vista, Água Fria e Areias. Seus graus variam de 8 a 11, com Afogados e Casa Amarela no topo (11). O tamanho dos nós reflete seu grau e as cores representam as microrregiões. Há forte presença da microrregião 5 (Afogados, San Martin, Curado, Areias), além de conexões centrais das microrregiões 3, 2, 1 e 4.

Como esses dez bairros pertencem a microrregiões distintas, suas conexões atuam como **corredores inter-regionais**, articulando partes diferentes da cidade e aumentando a integração estrutural da rede urbana.

9) Apresentação interativa do grafo

A visualização interativa do grafo exibe os 94 bairros como nós, conectados por arestas que representam ligações viárias. Três informações são codificadas diretamente no grafo:

- **Grau:** define o tamanho do nó — bairros mais conectados aparecem maiores;
- **Microrregião:** representada pela cor, facilitando a identificação de blocos territoriais;
- **Densidade da ego-subrede:** mostrada no tooltip junto do grau e da microrregião.

A busca por bairro permite localizar rapidamente qualquer nó: ao digitar parte do nome, ele recebe uma borda amarela destacada, o que ajuda a identificar seu papel na rede — se é periférico, corredor de passagem ou um hub local. Isso torna a ferramenta especialmente útil para análises direcionadas, como estudos de mobilidade ou planejamento urbano em bairros específicos.

O botão que destaca o caminho “**Nova Descoberta → Boa Viagem (Setúbal)**” evidencia, em laranja, o trajeto mínimo calculado pelo algoritmo de Dijkstra. O percurso obtido segue a sequência:

Nova Descoberta → Vasco da Gama → Morro da Conceição → Alto José do Pinho → Mangabeira → Tamarineira → Rosarinho → Encruzilhada → Torreão → Santo Amaro → Recife → Pina → Boa Viagem.

Parte 2:

Sobre o 2º dataset: Achemos um dataset (que nomeamos “bitcoinGraph.csv”) de notas que usuários dão uns para os outros dentro de uma plataforma de bitcoin e pensamos que poderíamos construir um grafo de relacionamento dos usuários, esse dataset o peso que seria a coluna “RATE” vem com pesos que variam de: 10 a -10, ou seja, ele possui notas negativas, então já meio que concluímos os requisitos para bellman-ford, porém dijkstra não funciona com pesos negativos entretanto nosso grafo é um grafo de relacionamento, quanto maior a nota maior o estreitamento das relações entre os

usuários então invertemos os pesos das relações para fazerem mais sentido com o nosso caso, por isso mesmo com um dataset com pesos negativos conseguimos utilizar dijkstra para calcular percurso.

1) Descrever o dataset

Colocamos todas as informações para descrever o dataset no em 2 json, em bitcoin.json temos ($|V|$, $|E|$, tipo: dirigido/ponderado,densidade) e em users.json temos a distribuição de graus do grafo dos usuários. Em users.json podemos ver alguns nós do grafo com 0 graus, o que a primeiro ponto parecia estranho, porém esse grafo é um grafo direcionado então esses grafos fazem parte do grafo, eles recebem arestas, mas não sai nenhuma deles.

2) Algoritmos

Após escolher e ajustar o dataset maior começamos a fazer o algoritmos o bfs /dfs (onde fizemos duas funções para cada um a primeira para rodar o algoritmo e mostrar o resultado e no segundo para rodar múltiplas vezes e mostrar ordem ,profundidade e ciclos), bellman-ford e dijkstra tínhamos alguns problemas resolvemos esses invertendo os pesos, mas colocamos uma verificação nas funções verificando se é pra normalizar os valores ou não justamente para não atrapalhar os testes onde dijkstra deveria travar com pesos negativos e bellman-ford deveria ter ciclos negativo, depois disso armazenamos os resultados em parte2_report.json, onde executamos dijkstra 5 vezes, bellman-ford uma vez com ciclo e uma sem ciclo , dfs e bfs a partir de 3 fontes diferentes, assim como manda dos requisitos do projeto.

3)Métricas de desempenho:

Para fazer as métricas de desempenho utilizamos o arquivo [cli.py](#) e la chamamos os algoritmos que foram feitos em [algorithms.py](#) para conseguirmos calcular o tempo e a memória para cada algoritmo e registramos tudo isso no parte2 report.json.

4) Visualização:

Nós optamos por fazer uma amostra do grafo, porém como nosso grafo tem mais de 27 mil conexões tentamos pegar em média umas 50 para fazer o sub grafo de uma forma que ficasse pelo menos um pouco conexo.

5) Discussão crítica:

Ficamos em dúvida de onde deixar as nossas conclusões então optamos por colocar junto do sub grafo onde colocamos de forma resumida sobre a nossa percepção de quando cada algoritmo era mais adequado

6) Testes

Os testes foram divididos da seguinte forma:

BFS([test_bfs.py](#)) - Validamos se o algoritmo calcula corretamente os níveis de distâncias a partir de um nó de origem. Construímos um grafo não-direcionado com arestas (1-2, 1-3, 2-4) e verificamos se o algoritmo atribui corretamente o nível 0 para a origem, nível 1 para vizinhos diretos e nível 2 para vizinhos de segundo grau.

DFS([test_dfs.py](#)) - Testamos a capacidade do algoritmo de detectar ciclos e classificamos as arestas retornadas, verificando se ele distingue corretamente entre arestas de árvore e arestas de retorno. Criamos um grafo direcionado contendo um ciclo(1→2→3→1) e uma ramificação (2→4) e o teste valida se a flag `has_cycle` torna verdadeira ao encontrar um nó já presente na pilha atual e verifica se o algoritmo diferencia as arestas de árvore(1→2) e arestas de retorno(3→1).

Dijkstra([test_dijkstra.py](#)) - Verificamos o cálculo do caminho mínimo em grafos com pesos positivos e a segurança do código contra entradas inválidas. Construímos um cenário de teste onde o caminho direto entre dois nós é mais caro que o caminho indireto (1→3 custa 5, enquanto 1→2→3 custa 3), garantindo que o algoritmo escolha corretamente a rota de menor custo. Além disso, validamos a robustez do algoritmo ao inserir propositalmente uma aresta com peso negativo, confirmando que ele lança uma exceção (`ValueError`).

Bellman-Ford([test_bellman_ford.py](#)) - Cobrimos os cenários críticos que diferenciam este algoritmo. Primeiro, testamos um grafo contendo uma aresta de peso negativo (-2) mas sem ciclos, validando se o algoritmo calcula corretamente o custo total reduzido (chegando a -1). Segundo, introduzimos um ciclo negativo intencional (1→2→3→1 com soma negativa) para confirmar se o algoritmo detecta o loop infinito das arestas e retorna a flag de erro (`-inf`), evitando que o programa entre em loop eterno.